# psgscoring

A Technical Handbook for Automated Polysomnography Scoring

Version 0.2.91 — April 2026

psgscoring contributors

https://github.com/bartromb/psgscoring

**Abstract**

This handbook provides a technical introduction to psgscoring, an open-source Python library (BSD-3) for AASM 2.6-compliant automated polysomnography (PSG) scoring. It is written for BSc-level computer science students with basic knowledge of Python, NumPy, and signal processing. The handbook covers the clinical context, signal processing pipeline, respiratory event detection algorithms, apnea type classification, and the architecture of the correction mechanisms that distinguish psgscoring from naïve threshold-based approaches.

**Prerequisites:** Python 3.9+, NumPy, SciPy, basic DSP (Fourier transform, filtering, envelopes).
**Repository:** https://github.com/bartromb/psgscoring
**Installation:** pip install psgscoring

## Contents

**1 Clinical Context: Why This Matters** **2**
1.1 Sleep Apnea . . . 2
1.2 The Scoring Problem . . . 2
1.3 The AASM Standard . . . 2

**2 Architecture Overview** **2**

**3 Signal Processing Chain** **3**
3.1 Step 1: Nasal Pressure Linearisation . . . 3
3.2 Step 2: Bandpass Filtering . . . 4
3.3 Step 3: Hilbert Envelope . . . 4
3.4 Step 4: Dynamic Baseline . . . 4
3.5 Step 5: Event Detection (Threshold Crossing) . . . 5
3.6 Step 6: MMSD Validation . . . 5

**4 Apnea Type Classification** **5**
4.1 Feature Extraction . . . 5
4.2 Decision Tree . . . 6
4.3 Code Example . . . 6

**5 $SpO_2$ Coupling** **7**

**6 Over-Counting Corrections** **7**
6.1 Fix 1: Post-Apnea Baseline Inflation . . . 7
6.2 Fix 2: $SpO_2$ Cross-Contamination . . . 7
6.3 Fix 3: Cheyne-Stokes Flagging . . . 7
6.4 Fix 4: Confidence Stratification . . . 7
6.5 Fix 5: Artefact-Flank Exclusion . . . 8
6.6 Fix 6: Local Baseline Validation . . . 8

**7 Under-Counting Corrections** **8**
7.1 Fix 7: Peak-Based Breath Detection . . . 8
7.2 Fix 8: Extended $SpO_2$ Window . . . 8
7.3 Fix 9: Flow Smoothing (removed in v0.2.91) . . . 8
7.4 Fix 10: Position Signal Auto-Mapping . . . 9
7.5 Fix 11: Configurable Scoring Profiles . . . 9
7.6 Fix 12: Flattening-Based RERA Detection . . . 9

**8 Breath-Amplitude Stability Filter (v0.2.91)** **9**
8.1 The Problem . . . 9
8.2 The Filter . . . 10
8.3 Calibration . . . 10

**9 AHI Confidence Interval (v0.2.91)** **10**
9.1 Mechanism . . . 10
9.2 Robustness Grade . . . 11
9.3 Clinical Example (SN2) . . . 11

**10 ECG-Derived Effort Classification** **11**
10.1 Spectral Effort Classifier . . . 11
10.2 Transformed ECG (TECG) . . . 11

**11 Arousal Detection** **12**
11.1 AASM Arousal Criteria . . . 12
11.2 Detection Algorithm . . . 12
11.3 RERA Detection . . . 12

**12 Snoring Analysis** **13**
12.1 Detection Method . . . 13
12.2 Output Metrics . . . 13

**13 Heart Rate Analysis** **13**
13.1 R-Peak Detection . . . 13
13.2 Output Metrics . . . 14
13.3 CVR Pattern . . . 14

**14 Body Position Analysis** **14**
14.1 Position Mapping . . . 14
14.2 Positional AHI . . . 15
14.3 Position-Induced Baseline Reset . . . 15

**15 Cheyne-Stokes Detection** **15**
15.1 Detection via Autocorrelation . . . 15
15.2 Event Flagging . . . 15

**16 Sleep Cycles and Architecture** **16**
16.1 Sleep Cycle Detection . . . 16
16.2 Stage-Specific Metrics . . . 16
16.3 Sleep Stage Transition Matrix . . . 16

**17 The Complete Pipeline** **16**

**18 External Validation: PSG-IPA** **17**
18.1 Results . . . 17
18.2 Ablation Analysis . . . 18
18.3 Key Findings . . . 18

**19 Testing** **18**
19.1 Unit Tests (37) . . . 18
19.2 Regression Tests (7) . . . 18
19.3 Property-Based Tests (3, Hypothesis) . . . 18

**20 Data Structures** **19**
20.1 Event Dictionary . . . 19
20.2 Summary Dictionary . . . 19

**21 Practical Tutorial: Processing an EDF File** **19**

**22 Design Decisions and Trade-offs** **20**
22.1 Why Rules, Not Deep Learning? . . . 20
22.2 Why Not Use a Single Threshold? . . . 20
22.3 Performance Characteristics . . . 21

**23 Breath Detection and Flattening Index** **21**
23.1 Breath Segmentation . . . 21
23.2 Flattening Index Computation . . . 21
23.3 Breath Amplitude for Hypopnea Detection . . . 22

**24 Periodic Limb Movement (PLM) Detection** **22**

**25 Signal Quality Assessment** **22**
25.1 Montage Plausibility . . . 22

**26 Common Pitfalls for Developers** **23**
26.1 Variable Scoping . . . 23
26.2 Docker Caching . . . 23
26.3 Shell Escaping . . . 23
26.4 EDF Limitations . . . 23
26.5 Translation Function Shadowing . . . 23

**27 Glossary** **24**

**28 Exercises** **24**

**29 References** **26**

# 1. Clinical Context: Why This Matters

## 1.1 Sleep Apnea

Obstructive Sleep Apnea (OSA) affects approximately 936 million adults worldwide. During sleep, the upper airway collapses partially or completely, causing breathing interruptions that fragment sleep and reduce blood oxygen levels. Left untreated, OSA increases the risk of hypertension, stroke, heart failure, and traffic accidents.

Diagnosis relies on **polysomnography** (PSG): an overnight recording of brain waves (EEG), eye movements (EOG), muscle tone (EMG), airflow, respiratory effort, blood oxygen saturation ($SpO_2$), body position, snoring, heart rhythm (ECG), and leg movements.

## 1.2 The Scoring Problem

A sleep technologist manually reviews the recording and counts:

- **Apneas**: ≥90% flow reduction for ≥10 seconds
- **Hypopneas**: ≥30% flow reduction for ≥10 seconds with ≥3% oxygen desaturation or an EEG arousal

The **Apnea-Hypopnea Index (AHI)** = (apneas + hypopneas) / hours of sleep, is the primary diagnostic metric:

| AHI | Severity | Treatment |
|---|---|---|
| <5 | Normal | None |
| 5–15 | Mild | Positional therapy, MAD |
| 15–30 | Moderate | CPAP |
| >30 | Severe | CPAP (urgent) |

Manual scoring takes 1–2 hours per study with 10–20% inter-scorer variability. `psgscoring` automates this process.

## 1.3 The AASM Standard

The American Academy of Sleep Medicine (AASM) publishes the *Manual for the Scoring of Sleep and Associated Events* (current: Version 2.6, 2020). This is the rulebook. Every threshold, every duration criterion, every exception in `psgscoring` traces back to a specific AASM rule. Key rules:

- **Rule 1A**: Hypopnea requires ≥3% $SpO_2$ desaturation
- **Rule 1B**: Hypopnea can alternatively be scored with an EEG arousal (without desaturation)
- Apneas are scored on the **thermistor** (oronasal thermal sensor)
- Hypopneas are scored on the **nasal pressure transducer**

# 2. Architecture Overview

`psgscoring` is organised as 12 independently importable submodules with a strict one-directional dependency graph:

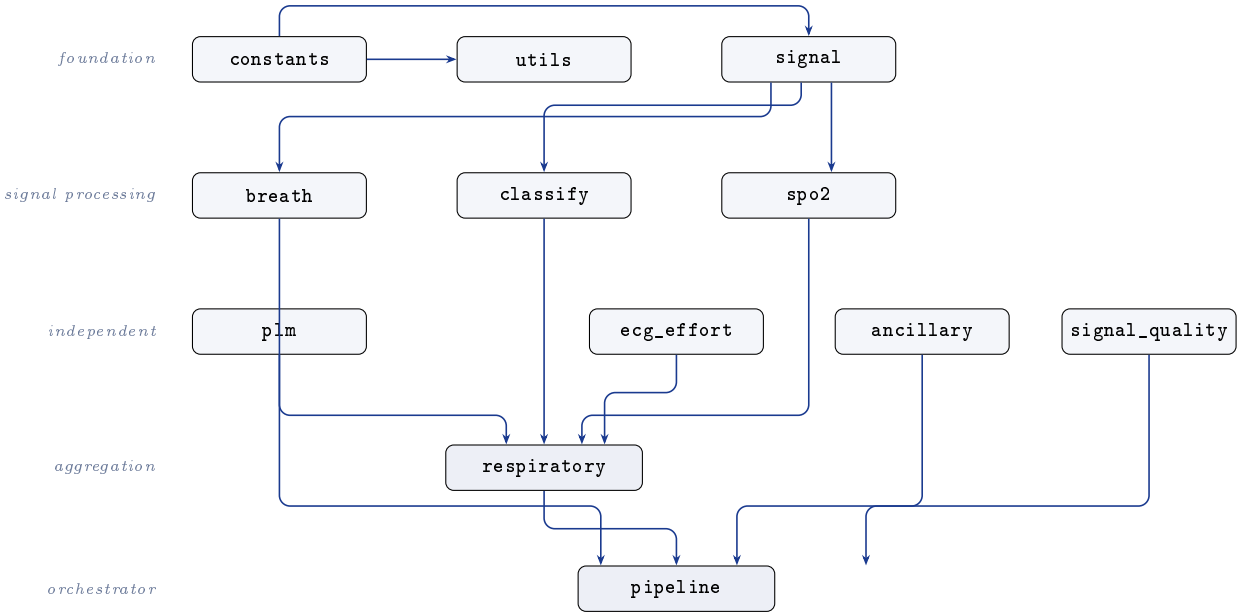



Figure 1: Module dependency graph. Arrows point from dependency to dependent module. The five layers correspond to increasing abstraction: foundation → signal processing → independent modules → aggregation → orchestrator.

Table 1: Submodule responsibilities and approximate size

| Module | Responsibility | LOC |
|---|---|---|
| `constants` | AASM thresholds, band limits, scoring profiles | 76 |
| `utils` | Sleep mask construction, channel detection | 134 |
| `signal` | Linearisation, baseline, MMSD validation | 327 |
| `breath` | Breath segmentation, flattening index | 254 |
| `classify` | 7-rule apnea type decision tree | 310 |
| `spo2` | $SpO_2$ coupling, ODI computation | 218 |
| `ecg_effort` | ECG-derived effort (TECG + spectral) | 180 |
| `plm` | Periodic Limb Movement detection | 271 |
| `ancillary` | Heart rate, snoring, position, CSR | 277 |
| `signal_quality` | Per-channel quality grading | 160 |
| `respiratory` | Apnea/hypopnea detection + 12 corrections | 1061 |
| `pipeline` | Master orchestrator | 420 |

## 3. Signal Processing Chain

The respiratory analysis begins with transforming the raw sensor data into a form suitable for event detection. This section describes each step mathematically and in code.

### 3.1 Step 1: Nasal Pressure Linearisation

The nasal pressure transducer measures *pressure*, not flow. The relationship between pressure and flow follows Bernoulli's equation:

$$\text{flow} \propto \sqrt{|\text{pressure}|} \tag{1}$$

The inverse square-root transform linearises the signal:

$$x_{\text{lin}}(t) = \text{sign}(x(t)) \cdot \sqrt{|x(t)|} \tag{2}$$

```
def linearise_nasal_pressure(signal):
    """Bernoulli square-root linearisation."""
    return np.sign(signal) * np.sqrt(np.abs(signal))
```

**Why this matters:** Without linearisation, the amplitude of hypopneas is systematically underestimated, because the pressure signal compresses peaks relative to the true flow.

### 3.2 Step 2: Bandpass Filtering

A 4th-order Butterworth bandpass filter (0.05–3 Hz) removes:

- DC drift and very slow baseline wander (<0.05 Hz)
- High-frequency noise, cardiac artefact (>3 Hz)

```
from scipy.signal import butter, sosfiltfilt

def bandpass_flow(signal, sf, low=0.05, high=3.0, order=4):
    sos = butter(order, [low, high], btype='band',
                 fs=sf, output='sos')
    return sosfiltfilt(sos, signal)
```

The normal respiratory rate is 12–20 breaths/minute (0.2–0.33 Hz), well within this passband.

### 3.3 Step 3: Hilbert Envelope

The instantaneous amplitude envelope is extracted via the analytic signal:

$$A(t) = |x(t) + i\mathcal{H}\{x(t)\}| \tag{3}$$

where $\mathcal{H}$ is the Hilbert transform. This provides a smooth, non-negative representation of breath amplitude.

```
from scipy.signal import hilbert

def compute_envelope(signal):
    analytic = hilbert(signal)
    return np.abs(analytic)
```

**Key insight:** The envelope "smooths over" individual breath cycles. This is useful for detecting apneas (amplitude $\to 0$) but can miss hypopneas where individual breath *peaks* are reduced but the envelope remains above threshold. This limitation motivates the peak-based detection added later (Section 7).

### 3.4 Step 4: Dynamic Baseline

The patient's "normal" breathing amplitude varies throughout the night (deeper in N3, shallower in REM, variable with position changes). A static threshold would fail. Instead, we compute a dynamic baseline:

$$B(t) = P_{95}\big(A(\tau) : \tau \in [t - 150\text{s}, t + 150\text{s}]\big) \tag{4}$$

A 5-minute sliding window, 95th percentile, advancing in 10-second steps with linear interpolation between anchor points.

```python
def compute_dynamic_baseline(envelope, sf,
                             window_s=300, step_s=10):
    n = len(envelope)
    anchors = range(0, n, int(step_s * sf))
    half_w = int(window_s * sf / 2)
    bl = np.zeros(n)
    for a in anchors:
        s = max(0, a - half_w)
        e = min(n, a + half_w)
        bl[a] = np.percentile(envelope[s:e], 95)
    # Interpolate between anchors
    return np.interp(np.arange(n),
                     list(anchors), bl[list(anchors)])
```

### 3.5 Step 5: Event Detection (Threshold Crossing)

An event is detected when the normalised amplitude drops below a threshold for ≥10 seconds:

$$\frac{A(t)}{B(t)} \leq 1 - \theta \quad \text{for} \quad \geq 10\text{s} \tag{5}$$

where $\theta = 0.90$ for apneas (thermistor) and $\theta = 0.30$ for hypopneas (nasal pressure). The normalised signal is:

$$\hat{x}(t) = \frac{A(t)}{B(t)} \tag{6}$$

Connected components below the threshold are found using `scipy.ndimage.label()`.

### 3.6 Step 6: MMSD Validation

Not every flat segment is a true apnea—some are artefacts (electrode disconnection). The Mean Magnitude of Second Derivative distinguishes them:

$$\text{MMSD} = \frac{1}{N} \sum_{i=2}^{N} |x''(i)| \tag{7}$$

True apneas have low but non-zero MMSD (residual cardiac oscillation); artefacts have MMSD $\approx 0$ (identically flat).

## 4. Apnea Type Classification

After detecting an event, we must determine its type: **obstructive** (airway blocked despite effort), **central** (no respiratory effort), or **mixed** (starts central, ends obstructive).

This is a clinical decision that depends on the thoraco-abdominal effort signals (RIP bands). A 7-rule decision tree processes the event:

### 4.1 Feature Extraction

For each event, the following features are computed from the thorax and abdomen RIP channels:

| Feature | Computation |
|---|---|
| `effort_ratio` | RMS(effort during event) / RMS(baseline) |
| `raw_var_ratio` | Variance(raw signal during event) / Variance(baseline) |
| `paradox_corr` | Pearson correlation(thorax, abdomen) — negative = paradoxical |
| `phase_angle_deg` | Mean $\lvert\Delta\phi\rvert$ via Hilbert phase of each RIP channel |
| `quarter_efforts` | Effort ratio for each quarter of the event duration |
| `first_ratio, second_ratio` | Effort ratio for first vs. second half |
| `flattening_index` | Mean inspiratory flattening of overlapping breaths |

## 4.2 Decision Tree

Table 2: Classification rules (evaluated in order; first match wins)

| Rule | Condition | Type | Confidence |
|---|---|---|---|
| 0 | Phase angle $\geq 45°$ | Obstructive | 0.75–0.97 |
| 1 | Paradoxical + high raw variability | Obstructive | 0.85–0.95 |
| 2 | High variability, low envelope | Obstructive | 0.70–0.85 |
| 3 | First half low effort, second half high | Mixed | 0.70–0.95 |
| 4 | Clear effort present | Obstructive | 0.65–0.80 |
| 5 | Truly flat, no effort, no paradox | Central | 0.70–0.90 |
| 5a | Low effort (gray zone), no obstructive evidence | Central | 0.45–0.75 |
| 5b | ECG confirms no inspiratory effort (TECG) | Central | 0.75–0.85 |
| 6 | Low effort default → central; else → obstructive | varies | <0.60 |

**The cardiac pulsation problem:** During a true central apnea, the heart continues to beat. These mechanical oscillations appear on the thorax/abdomen RIP bands as low-amplitude periodic signals. A naïve algorithm interprets this as "effort present" → obstructive. Rules 5, 5a, and 5b are specifically designed to handle this, using relaxed thresholds and ECG-derived effort analysis.

## 4.3 Code Example

```python
from psgscoring import classify_apnea_type

typ, confidence, detail = classify_apnea_type(
    onset_idx=start_sample,
    end_idx=end_sample,
    thorax_env=thorax_envelope,
    abdomen_env=abdomen_envelope,
    thorax_raw=thorax_raw_signal,
    abdomen_raw=abdomen_raw_signal,
    effort_baseline=effort_bl,
    sf=256,
    flattening_index=0.35,  # from breath analysis
)
# typ: "obstructive", "central", or "mixed"
# confidence: 0.0 - 1.0
# detail: {"rule_index": 0, "decision_reason": "..."}
```

## 5. $SpO_2$ Coupling

AASM Rule 1A requires a ≥3% oxygen desaturation associated with each hypopnea. The coupling algorithm:

1. Compute the $SpO_2$ baseline: 90th percentile during sleep
2. For each candidate hypopnea, search for the nadir in a **post-event window** (configurable: 30–45 seconds)
3. Desaturation = pre-event $SpO_2$ − nadir
4. If desaturation ≥ 3%: hypopnea confirmed (Rule 1A)

**Circulatory delay:** The finger pulse oximeter has a 20–40 second delay relative to the lungs. A 30-second window misses nadirs arriving at 30–45 seconds. The *standard* scoring profile uses 45 seconds; *strict* uses 30 seconds.

## 6. Over-Counting Corrections

Naïve threshold-based scoring systematically *inflates* AHI. Six mechanisms are identified and corrected:

### 6.1 Fix 1: Post-Apnea Baseline Inflation

After an apnea, the patient hyperventilates (compensatory breathing). These large recovery breaths enter the 5-minute baseline window, inflating it. Subsequent normal breaths then appear "reduced" relative to the inflated baseline → false hypopneas.

**Solution:** Exclude a 30-second recovery mask after each apnea from the baseline computation.

```
# Pseudocode
for apnea in detected_apneas:
    mask[apnea.end : apnea.end + 30*sf] = True
baseline = percentile_95(envelope[~mask], window=300s)
```

### 6.2 Fix 2: $SpO_2$ Cross-Contamination

At AHI > 60/h, events follow each other so quickly that the $SpO_2$ nadir of event $N$ falls within the search window of event $N+1$. Event $N+1$ gets an undeserved desaturation → false hypopnea.

**Solution:** Flag events where the preceding event's nadir window overlaps. Since v0.8.14, this is informational only (desaturation is always computed).

### 6.3 Fix 3: Cheyne-Stokes Flagging

Cheyne-Stokes respiration (CSR) produces a crescendo-decrescendo pattern. Each trough can cross the 30% threshold → scored as hypopnea. But these are not independent events; they are one central breathing pattern.

**Solution:** Detect periodicity via autocorrelation. Events whose inter-event interval matches the CSR cycle (±12 s) are flagged as `csr_flagged`.

### 6.4 Fix 4: Confidence Stratification

When RIP signal quality is poor, Rule 6 defaults to "obstructive." This inflates OAHI.

**Solution:** Report events at four confidence tiers (≥0.85, ≥0.60, ≥0.40, all). The PDF report shows OAHI at each threshold.

### 6.5 Fix 5: Artefact-Flank Exclusion

After a flat-line gap (electrode disconnection), the recovery ramp can cross the flow-reduction threshold.

**Solution:** Append a 15-second exclusion window after each flat-line segment ≥10 seconds.

### 6.6 Fix 6: Local Baseline Validation

The rolling baseline can be inflated by recovery hyperpnea. Normal breathing then appears reduced → false-positive hypopneas of 60–80+ seconds.

**Solution:** Compare the event amplitude with the 30-second pre-event window. If the local reduction is <20%, the event is rejected. Additionally (v0.2.91), a *stability-aware threshold* raises this to 30% when the surrounding breathing is stable (envelope CV <0.30 in a ±120 s window), preventing normal variability from passing the local check.

```
def _validate_local_reduction(envelope, onset, end, sf):
    pre_start = max(0, onset - int(30 * sf))
    pre_mean = np.mean(envelope[pre_start:onset])
    event_mean = np.mean(envelope[onset:end])
    reduction = 1 - event_mean / pre_mean
    min_reduction = 0.20  # default
    # v0.2.91: stability-aware threshold
    stab_seg = envelope[onset-120*sf : end+120*sf]
    if len(stab_seg) > 10*sf:
        cv = np.std(stab_seg) / np.mean(stab_seg)
        if cv < 0.30:
            min_reduction = 0.30  # stricter for stable
    return reduction >= min_reduction, reduction
```

This complements the post-detection stability filter (Section 8), which operates on per-breath amplitudes rather than the continuous envelope.

## 7. Under-Counting Corrections

The complementary problem: the algorithm *misses* events that a human scorer would detect.

### 7.1 Fix 7: Peak-Based Breath Detection

AASM defines hypopnea as "peak signal excursions drop ≥30%." The Hilbert envelope smooths over breath cycles, missing subtle per-breath reductions.

**Solution:** Detect individual breaths via zero-crossings, compute per-breath peak amplitude, and use a combined mask (envelope OR peak-based).

**Consecutive breath requirement (v0.2.91):** To prevent isolated reduced breaths from triggering false events, peak detection now requires ≥`PEAK_MIN_CONSECUTIVE_BREATHS` consecutive reduced breaths (default: 3 for strict/standard, 2 for sensitive). This replaced flow smoothing as the anti-fragmentation mechanism.

### 7.2 Fix 8: Extended $SpO_2$ Window

Finger oximetry has 20–40 s circulatory delay. The 30 s window misses late nadirs.

**Solution:** Extend to 45 s (configurable per scoring profile).

### 7.3 Fix 9: Flow Smoothing (removed in v0.2.91)

Brief oscillations above the 70% threshold can fragment a long event into sub-10 s segments that are each discarded. Previous versions applied a 3-second rolling mean before thresholding.

**Note on categorisation:** Fix 9 was originally implemented as an under-counting correction (preventing event fragmentation). However, ablation analysis on PSG-IPA (Section 18) demonstrated that it was, in practice, the *dominant source of over-counting*: the smoothing bridged brief recovery breaths into continuous "reduced flow" segments, generating +54 false hypopneas (+11.2/h) on a mild-OSA recording. This finding — that a fix designed to reduce under-counting actually caused over-counting — illustrates why empirical validation against multi-scorer reference data is essential.

**Status (v0.2.91):** Flow smoothing has been **removed from all profiles**. The consecutive-breath requirement (`PEAK_MIN_CONSECUTIVE_BREATHS` $\geq 3$, see Fix 7) now replaces smoothing as the anti-fragmentation mechanism.

### 7.4 Fix 10: Position Signal Auto-Mapping

Some EDF systems store position as raw ADC values (0–255) instead of standard codes (0–4).

**Solution:** Auto-detect encoding; apply percentile-based quantisation.

### 7.5 Fix 11: Configurable Scoring Profiles

Table 3: Three predefined scoring profiles (v0.2.91)

| Profile | Threshold | $SpO_2$ win. | Smooth | Peak det. | Stab. filter | Use case |
|---|---|---|---|---|---|---|
| Strict | ≥30% | 30 s | None | Envelope only | CV<0.45 | Research |
| Standard | ≥30% | 45 s | None | Peak + envelope | CV<0.45 | Clinical (AASM 2.6) |
| Sensitive | ≥25% | 45 s | None | Peak + envelope | CV<0.45 | UARS / screening |

**Note on smoothing (v0.2.91):** Flow smoothing (previously 3 s for standard, 5 s for sensitive) was removed from all profiles after ablation analysis on PSG-IPA demonstrated that 3-second smoothing was the dominant source of over-counting, adding +54 false hypopneas to a single mild-OSA recording (Section 18).

### 7.6 Fix 12: Flattening-Based RERA Detection

RERAs (Respiratory Effort-Related Arousals) are flow-limited segments without amplitude reduction, terminated by an arousal. The flattening index measures the fraction of the inspiratory segment exceeding 80% of peak amplitude:

$$FI = \frac{|\{t : x(t) > 0.8 \cdot x_{\max}\}|}{|\{t : x(t) > 0\}|} \tag{8}$$

Normal triangular waveform: $FI \approx 0.10$. Flattened (flow limitation): $FI > 0.30$.

## 8. Breath-Amplitude Stability Filter (v0.2.91)

A key challenge for automated scoring is distinguishing genuine flow reduction from normal breath-to-breath amplitude variability. Human scorers resolve this intuitively by comparing the candidate event with the surrounding breathing pattern.

### 8.1 The Problem

At low event density (AHI <5), the P95 rolling baseline tracks the highest breaths. Normal variability (tidal volume fluctuations) then generates apparent ≥30% reductions that are scored as false hypopneas. This is not a deficiency of the P95 approach per se but a consequence of applying a relative threshold to a signal with inherent biological variability.

### 8.2 The Filter

For each hypopnea candidate, the system examines the peak amplitude of individual breaths in a ±120-second window:

1. Collect all detected breaths (via zero-crossing segmentation) within ±120 s of the event midpoint.
2. Compute the coefficient of variation (CV) of breath amplitudes: $\mathrm{CV} = \sigma_A/\mu_A$.
3. If CV $< 0.45$ and $\geq 10$ breaths are available, reject the candidate as normal variability.

```
# Stability filter pseudocode
for event in hypopnea_candidates:
    breaths_nearby = get_breaths(
        event.mid - 120s, event.mid + 120s)
    if len(breaths_nearby) >= 10:
        cv = std(amplitudes) / mean(amplitudes)
        if cv < 0.45:
            reject(event, reason="stable_breathing")
```

### 8.3 Calibration

The CV threshold of 0.45 was calibrated on PSG-IPA (Section 18):

Table 4: Stability filter calibration across severity spectrum

| PSG | Median CV | Rejected | ΔAHI | Clinical effect |
|---|---|---|---|---|
| SN4 (Normal) | 0.428 | 56 | −68% | Corrects false Mild → Normal |
| SN2 (Normal) | — | 61 | −57% | Reduces over-counting |
| SN1 (Mild) | 0.478 | 4 | −4% | Minimal (true events preserved) |
| SN3 (Severe) | 0.733 | 11 | −3% | Minimal |

The filter's severity-dependent selectivity confirms that it targets normal variability (low CV) rather than true pathology (high CV).

## 9. AHI Confidence Interval (v0.2.91)

Every AHI carries inherent uncertainty. Rather than hiding this behind a single number, the system makes it explicit.

### 9.1 Mechanism

Every analysis automatically runs all three scoring profiles and reports the AHI as an interval:

```
results = run_pneumo_analysis(raw, hypno,
                              scoring_profile="standard")
iv = results["ahi_interval"]
print(f"AHI: {iv['standard']['ahi']}")
print(f"Interval: [{iv['interval'][0]} - "
      f"{iv['interval'][1]}]")
print(f"Robustness: {iv['robustness_grade']}")
# Output: AHI: 8.1
#         Interval: [4.0 - 8.1]
#         Robustness: B
```

### 9.2 Robustness Grade

A grade summarises severity concordance across profiles:

Table 5: Robustness grading

| Grade | Criterion | Clinical interpretation |
|---|---|---|
| A | All 3 profiles same severity | Robust — treatment unambiguous |
| B | 2/3 concordant | Probable — clinical correlation recommended |
| C | All discordant | Uncertain — manual review recommended |

### 9.3 Clinical Example (SN2)

SN2 demonstrates why the interval matters. Twelve certified technologists scored this recording: 8 said Normal, 4 said Mild (AHI range 0.2–6.6). The algorithm gives:

| | | |
|---|---|---|
| Strict: | 8.2 (Mild) | |
| Standard: | 9.3 (Mild) | |
| Sensitive: | 5.4 (Mild) | ← within scorer IQR |

Grade A ("Robust Mild") — but the interval [5.4–9.3] immediately shows the AHI hovers near the 5/h threshold. This is precisely the clinical situation where a single number ("AHI = 9.3, Mild") would mislead the clinician into a CPAP trial, while the interval communicates that watchful waiting may be equally appropriate.

## 10. ECG-Derived Effort Classification

The routinely recorded ECG contains information about respiratory effort — if you know where to look.

### 10.1 Spectral Effort Classifier

During a true central apnea, the RIP signal should show only cardiac pulsation. The spectral classifier computes PSD of the RIP signal during each event:

- Cardiac band: 0.8–2.5 Hz (adaptive per patient HR)
- Respiratory band: 0.1–0.5 Hz

If >75% of power is in the cardiac band and <20% in respiratory → flag as probable central.

### 10.2 Transformed ECG (TECG)

Following Berry et al. (JCSM 2019):

1. High-pass filter ECG at 30 Hz
2. Blank QRS complexes (±50 ms around R-peaks)
3. The residual reveals inspiratory EMG bursts from intercostal muscles
4. If no bursts detected → no respiratory effort → central

```
from psgscoring.ecg_effort import ecg_effort_assessment

result = ecg_effort_assessment(
    ecg_data, thorax_raw, abdomen_raw,
    sf_ecg=256, sf_effort=256,
    onset_idx=start, end_idx=end,
    r_peaks=r_peak_indices
)
# result["reclassify_as_central"]: bool
```

```
# result["ecg_effort_present"]: bool
```

## 11. Arousal Detection

An **arousal** is a brief awakening from sleep (3–15 seconds of alpha/beta EEG activity) that fragments sleep architecture without the patient becoming fully conscious. Arousals are clinically important because they cause daytime sleepiness even when AHI is low.

### 11.1 AASM Arousal Criteria

The AASM defines an arousal as:

- An abrupt shift in EEG frequency to alpha, theta, or >16 Hz (not spindles)
- Duration ≥3 seconds
- Preceded by ≥10 seconds of stable sleep
- During REM: requires concurrent increase in chin EMG

The **Arousal Index (AI)** = total arousals / hours of sleep. Normal: <15/h. The AI is further broken down:

- **Respiratory arousals**: occurring within 3 s of a respiratory event termination
- **PLM-associated arousals**: occurring within 0.5 s of a leg movement
- **Spontaneous arousals**: all remaining

### 11.2 Detection Algorithm

YASAFlaskified implements a two-phase arousal detection in `arousal_analysis.py`:

**Phase 1 — EEG-based detection:**

1. Compute spectral power in the alpha (8–12 Hz) and beta (>16 Hz) bands using a 1-second sliding window
2. Detect segments where alpha+beta power exceeds a dynamic threshold (mean + 2×SD of the surrounding 30-second epoch)
3. Merge segments separated by <1 s; discard segments <3 s
4. During REM: require concurrent EMG amplitude increase (>2× baseline chin EMG RMS)

**Phase 2 — Filtering and coupling:**

1. **K-complex exclusion**: if the first 1 s of a candidate contains a negative peak $<-75$ $\mu$V followed by a positive peak >30 $\mu$V, the minimum duration is raised from 3 s to 5 s (prevents scoring the trailing alpha rebound as a standalone arousal)
2. **CVR confidence boost**: if the ECG shows a bradycardia during the event followed by sympathetic tachycardia (≥10 bpm rise within 15 s), a confidence boost of 0.10–0.20 is added
3. **Respiratory coupling**: arousals within 3 s of a respiratory event termination are tagged as respiratory arousals
4. **Rule 1B reinstatement**: flow reductions with an arousal but without ≥3% desaturation are reinstated as hypopneas (AASM Rule 1B)

**Why K-complex exclusion matters:** K-complexes are normal NREM phenomena that produce a transient power increase in arousal-detection bands. Without this filter, up to 15% of detected "arousals" are actually K-complexes — inflating the arousal index and triggering false Rule 1B reinstatements.

### 11.3 RERA Detection

RERAs (Respiratory Effort-Related Arousals) are detected from two sources:

1. **FRI-RERA**: flow reductions (≥30%, ≥10 s) without desaturation, coupled to an arousal within 15 s of event termination
2. **Flattening-RERA**: sequences of ≥3 consecutive breaths with flattening index >0.30 spanning ≥10 s, terminated by an arousal, *without* meeting hypopnea amplitude criteria

$$\text{RDI} = \text{AHI} + \frac{n_{\text{FRI-RERA}} + n_{\text{flat-RERA}}}{\text{TST (hours)}} \tag{9}$$

The RDI is clinically relevant for Upper Airway Resistance Syndrome (UARS), where AHI may be normal but the patient presents with excessive daytime sleepiness.

## 12. Snoring Analysis

Snoring is detected from the microphone or vibration sensor channel (typically labelled "Snore" or "Phono" in the EDF).

### 12.1 Detection Method

1. Bandpass filter the snore channel (30–300 Hz) to isolate snoring frequencies
2. Compute the RMS amplitude in 1-second windows
3. Determine a dynamic snore threshold: the 60th percentile of the RMS amplitude during sleep epochs (P60)
4. Segments where RMS > P60 for ≥3 consecutive seconds are classified as snoring epochs

### 12.2 Output Metrics

| Metric | Definition |
|---|---|
| Snore duration (min) | Total minutes of detected snoring |
| Snore % of TST | Fraction of sleep time with snoring |
| Snore index (/h) | Snoring episodes per hour of sleep |

```python
# Access snoring results
snore = results["snore"]["summary"]
print(f"Snoring: {snore['snore_pct_tst']:.1f}% of sleep")
print(f"Snore index: {snore['snore_index']:.1f}/h")
```

**Clinical note:** Snoring alone is not diagnostic, but a high snore index combined with mild AHI may indicate UARS. The position overlay in the PDF report shows whether snoring is position-dependent (common: supine-dominant snoring).

## 13. Heart Rate Analysis

Heart rate (HR) is derived from the ECG channel via R-peak detection.

### 13.1 R-Peak Detection

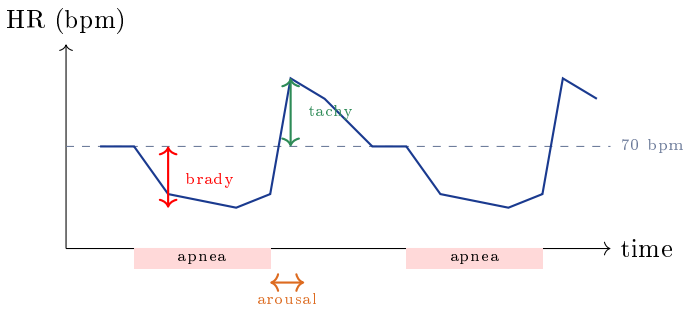
1. Bandpass filter ECG (5–30 Hz, 4th-order Butterworth) to enhance QRS complexes
2. Apply `scipy.signal.find_peaks()` with adaptive height threshold (0.6× 90th percentile of filtered signal) and minimum distance (0.3 s = 200 bpm maximum)
3. Compute R-R intervals; reject outliers (<0.3 s or >2.0 s)
4. Instantaneous HR = 60 / R-R interval (bpm)

### 13.2 Output Metrics

| Metric | Unit | Clinical significance |
|---|---|---|
| Mean HR | bpm | Normal: 60–100 during sleep |
| Min HR | bpm | Bradycardia <50; check medication (beta-blockers) |
| Max HR | bpm | Tachycardia >100; may indicate autonomic arousal |
| Bradycardia episodes | count | Number of segments <50 bpm ≥10 s |
| Tachycardia episodes | count | Number of segments >100 bpm ≥10 s |

### 13.3 CVR Pattern

The Cyclic Variation of heart Rate (CVR) is a characteristic pattern in OSA: *bradycardia* during the apnea (vagal tone from hypoxia), followed by *tachycardia* upon arousal (sympathetic activation). The magnitude of this swing (ΔHR ≥10 bpm within 15 s) is used as a confidence boost for arousal detection (Section 11).




## 14. Body Position Analysis

Body position during sleep determines whether OSA is *position-dependent* — a clinically important distinction because position-dependent OSA may be treated with positional therapy (e.g., tennis ball, NightBalance device) instead of CPAP.

### 14.1 Position Mapping

The EDF position channel uses one of three encoding schemes:

| Encoding | Format | Systems |
|---|---|---|
| Integer codes | 0=supine, 1=left, 2=prone, 3=right, 4=upright | Standard |
| Raw ADC | 0–255 (voltage proportional to tilt angle) | SomnoMedics, Embla |
| String labels | "Supine", "Left", etc. | Some Natus systems |

The auto-mapping function (`_map_position_signal()`) detects the encoding and applies percentile-based quantisation for raw ADC values:

```
# Automatic position mapping
pos = results["position"]["summary"]
for position in ["supine", "left", "right", "prone", "upright"]:
    pct = pos.get(f"pct_{position}", 0)
    ahi = pos.get(f"ahi_{position}", 0)
    print(f"  {position:8s}: {pct:.0f}% of TST, AHI={ahi:.1f}/h")
```

### 14.2 Positional AHI

Events are mapped to body position via the position channel at event onset. The position-specific AHI is:

$$\mathrm{AHI}_{\mathrm{pos}} = \frac{n_{\text{events in position}}}{\text{time in position (hours)}} \tag{10}$$

**Supine-dominant OSA** is defined as supine AHI $\geq 2\times$ non-supine AHI. This phenotype is more prevalent in younger, non-obese patients and responds well to positional therapy.

### 14.3 Position-Induced Baseline Reset

When a patient changes position (e.g., supine $\rightarrow$ lateral), the respiratory amplitude changes abruptly. The dynamic baseline must reset to avoid false events at the transition. `psgscoring` detects position changes and resets the baseline window at each transition (v0.8.25).

## 15. Cheyne-Stokes Detection

Cheyne-Stokes Respiration (CSR) is a crescendo-decrescendo breathing pattern characteristic of central sleep apnea, most commonly seen in heart failure patients.

### 15.1 Detection via Autocorrelation

The flow envelope is analysed using autocorrelation to detect periodic breathing patterns:

1. Compute the autocorrelation of the Hilbert envelope over a 5-minute window
2. Search for a secondary peak between 40 and 100 seconds (typical CSR cycle length: 45–75 seconds)
3. If the peak-to-trough ratio exceeds a threshold (1.5× the surrounding baseline), CSR is detected
4. Determine the CSR periodicity from the peak location

$$R(\tau) = \frac{1}{N} \sum_{t=0}^{N-\tau} A(t) \cdot A(t+\tau) \tag{11}$$

### 15.2 Event Flagging

After CSR detection, individual respiratory events whose inter-event interval matches the detected periodicity ($\pm 12$ s) are retroactively flagged:

```python
# CSR results
csr = results["respiratory"]["summary"]
print(f"CSR detected: {csr.get('csr_detected', False)}")
print(f"CSR period:   {csr.get('csr_period_s', 0):.0f}s")
print(f"Events flagged: {csr.get('n_csr_flagged', 0)}")
print(f"AHI without CSR: {csr.get('ahi_csr_corrected', 0):.1f}/h")
```

**Clinical impact:** CSR requires fundamentally different treatment (Adaptive Servo-Ventilation, ASV) rather than standard CPAP. Without CSR flagging, a heart failure patient may receive an AHI >60/h and be started on CPAP, which may worsen central apneas. The corrected AHI (excluding CSR-flagged events) helps the clinician distinguish obstructive from central pathology.

## 16. Sleep Cycles and Architecture

While sleep staging is performed by YASA (upstream of `psgscoring`), the platform computes additional sleep architecture metrics.

### 16.1 Sleep Cycle Detection

Sleep cycles are detected following Feinberg & Floyd (1979) criteria:

- A NREM period requires ≥15 minutes of continuous NREM sleep (N1, N2, or N3)
- REM consolidation: REM segments separated by <2 minutes of NREM are merged into a single REM period
- A cycle = one NREM period + the subsequent REM period
- Normal sleep: 4–6 cycles per night, each 90–120 minutes

### 16.2 Stage-Specific Metrics

Sleep architecture affects respiratory scoring because:

- N3 (deep sleep): respiratory amplitude is lower → the dynamic baseline must be stage-blended
- REM: muscle atonia increases upper airway collapsibility → REM-AHI is typically higher than NREM-AHI
- REM-dominant OSA (REM-AHI/NREM-AHI >2) is a distinct phenotype, more common in women and younger patients

```python
# Stage-specific AHI
resp = results["respiratory"]["summary"]
print(f"REM AHI:  {resp.get('ahi_rem', '?')}/h")
print(f"NREM AHI: {resp.get('ahi_nrem', '?')}/h")
ratio = resp.get('ahi_rem', 0) / max(resp.get('ahi_nrem', 1), 1)
if ratio > 2:
    print("REM-dominant OSA phenotype")
```

### 16.3 Sleep Stage Transition Matrix

The PDF report includes a 5×5 transition matrix showing the number of stage transitions (W/N1/N2/N3/R → W/N1/N2/N3/R). This helps identify:

- Excessive W→N1 transitions: fragmented sleep onset
- High N2→W count: frequent arousals from light sleep
- Low N2→N3 transitions: reduced deep sleep initiation
- Absent or rare transitions to R: REM suppression (medication, depression, severe OSA)

## 17. The Complete Pipeline

The master function `run_pneumo_analysis()` orchestrates all modules:

```python
from psgscoring.pipeline import run_pneumo_analysis

results = run_pneumo_analysis(
    raw_pneumo,          # MNE Raw object
    hypno,               # ["W","N1","N2",...]
    channel_map,         # {"flow": "Pressure", ...}
    scoring_profile="standard",
)
# results["respiratory"]["events"]: list of dicts
# results["respiratory"]["summary"]["ahi_total"]
```

```
# results["spo2"]["summary"]["odi_3pct"]
# results["plm"]["summary"]["plmi"]
```

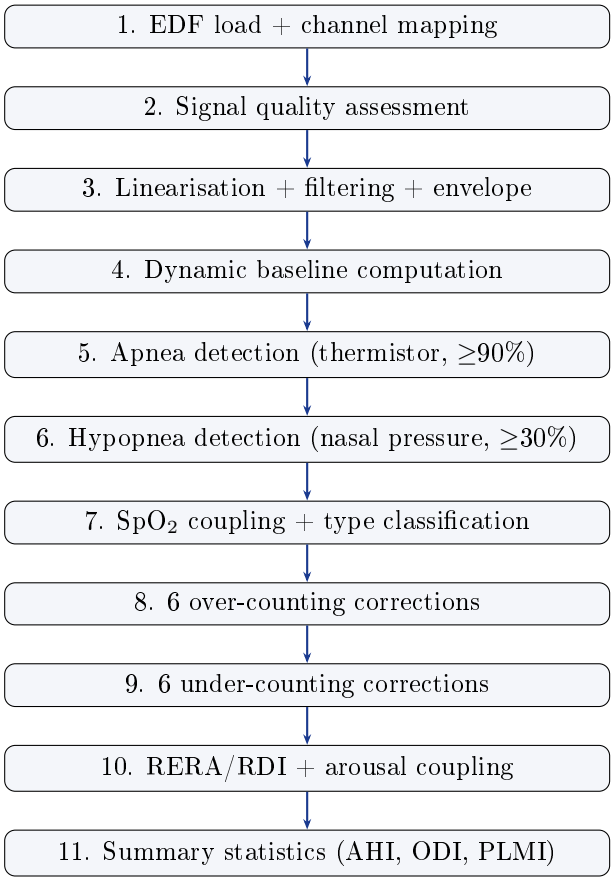



Figure 2: Processing pipeline steps

## 18. External Validation: PSG-IPA

`psgscoring` v0.2.91 was validated on the PSG Inter-scorer reliability Project on Apneas (PSG-IPA), an open-access dataset on PhysioNet containing 5 diagnostic PSG recordings each scored by up to 12 independent certified sleep technologists (59 scorer sessions total).

### 18.1 Results

Table 6: AHI agreement: `psgscoring` (standard profile, clinical hypnogram) vs. human scorer median on PSG-IPA

| **PSG** | $n$ | $\mathbf{AHI_{med}}$ | $\mathbf{AHI_{algo}}$ | $\lvert\Delta\rvert$ | **Severity** |
|---|---|---|---|---|---|
| SN1 | 11 | 6.0 | 8.1 | 2.1 | Mild |
| SN2 | 12 | 3.7 | 9.3 | 5.6 | Mild / Normal* |
| SN3 | 12 | 54.0 | 53.8 | 0.2 | Severe |
| SN4 | 12 | 3.5 | 4.3 | 0.8 | Normal |
| SN5 | 12 | 9.9 | 11.4 | 1.5 | Mild |
| Mean | | | | 2.0 | Concordance: 4/5 |

*Scorers split 8/12 Normal, 4/12 Mild. Sensitive profile (5.4) within scorer range.

### 18.2 Ablation Analysis

Systematic parameter-by-parameter analysis on SN1 (11 scorers, median AHI 6.0) identified 3-second flow smoothing as the dominant source of over-counting:

Table 7: Ablation: which parameter causes over-counting?

| Configuration | Hyp | AHI | Effect |
|---|---|---|---|
| Strict baseline (envelope only) | 12 | 4.7 | — |
| + Peak detection | 36 | 8.8 | +23 events |
| + 3-second smoothing | 76 | 15.9 | +54 events |
| *Human scorers (median)* | *∼15* | *6.0* | |

### 18.3 Key Findings

1. **Flow smoothing was removed** from all profiles — it was the single largest source of over-counting (+11.2/h).
2. **Stability filter** (CV<0.45) reduced over-counting by 57–68% in normal patients while preserving accuracy in severe OSA (−3% on SN3).
3. **AHI confidence interval** correctly identified the borderline case (SN2) where human scorers disagreed.

## 19. Testing

`psgscoring` includes 47 tests (Python 3.9–3.12 CI):

### 19.1 Unit Tests (37)

Each module has dedicated tests covering edge cases: signal processing with zero-length arrays, classification with extreme feature values, $SpO_2$ coupling at boundary conditions.

### 19.2 Regression Tests (7)

Synthetic signals with known ground truth:

- Obstructive apnea: effort present + flow absent → must classify obstructive
- Central apnea: no effort + no flow → must classify central
- Mixed apnea: first half central, second half obstructive
- Dynamic baseline stability on constant-amplitude input
- $SpO_2$ desaturation detection with known nadir
- Breath count for 12 breaths/minute signal
- Flattening index passthrough to classification detail

### 19.3 Property-Based Tests (3, Hypothesis)

Random inputs to `classify_apnea_type()`:

- 500 random combinations: never crashes, always returns valid type
- Low effort inputs: must not produce high-confidence obstructive
- Short segments (2–100 samples): crash-free

```
# Running tests
cd psgscoring
python -m pytest tests/ -v
```

# 20. Data Structures

## 20.1 Event Dictionary

Each respiratory event is represented as a Python dictionary:

```
{
    "type":                "obstructive",  # or "central"/"mixed"/"hypopnea"
    "onset_s":             1234.5,         # seconds from recording start
    "duration_s":          18.0,
    "stage":               "N2",           # sleep stage at event onset
    "desaturation_pct":    4.2,            # SpO2 drop (Rule 1A)
    "min_spo2":            89.3,
    "confidence":          0.82,
    "rule_index":          1,              # which classification rule fired
    "flow_reduction":      0.45,           # fraction of baseline
    "classify_detail": {
        "effort_ratio":     0.72,
        "raw_var_ratio":    0.85,
        "phase_angle_deg":  52.3,
        "decision_reason":  "paradox_movement",
        "flattening_index": 0.28,
    },
    "spo2_cross_contaminated": False,
    "csr_flagged":         False,
}
```

## 20.2 Summary Dictionary

```
{
    "ahi_total":        29.1,
    "oahi":             28.5,
    "cahi":             0.6,
    "n_obstructive":    85,
    "n_central":        2,
    "n_mixed":          1,
    "n_hypopnea":       78,
    "n_csr_flagged":    0,
    "n_local_baseline_rejected": 42,
    "severity":         "moderate",
}
```

# 21. Practical Tutorial: Processing an EDF File

A minimal end-to-end example:

```
import mne
from psgscoring.pipeline import run_pneumo_analysis
from psgscoring.utils import build_sleep_mask

# 1. Load EDF
raw = mne.io.read_raw_edf("patient.edf", preload=True)
```

```python

# 2. Assume hypnogram is available (from YASA or manual)
hypno = ["W","W","N1","N2","N2","N3", ...]  # per 30-s epoch

# 3. Define channel mapping
channel_map = {
    "flow":            "Pressure",       # nasal pressure
    "flow_thermistor":"Flow",            # oronasal thermistor
    "thorax":          "RIP Thora",
    "abdomen":         "RIP Abdom",
    "spo2":            "SpO2",
    "snore":           "Snore",
    "position":        "Pos.",
    "ecg":             "ECG II",
    "plm_l":           "PLMl",
    "plm_r":           "PLMr",
}

# 4. Run analysis
results = run_pneumo_analysis(
    raw, hypno, channel_map,
    scoring_profile="standard",
)

# 5. Extract results
resp = results["respiratory"]
print(f"AHI:  {resp['summary']['ahi_total']:.1f}/h")
print(f"OAHI: {resp['summary']['oahi']:.1f}/h")
print(f"Events: {len(resp['events'])}")

for ev in resp["events"][:5]:
    print(f"  {ev['type']:12s} at {ev['onset_s']:.0f}s, "
          f"dur={ev['duration_s']:.0f}s, "
          f"conf={ev['confidence']:.2f}")
```

## 22. Design Decisions and Trade-offs

### 22.1 Why Rules, Not Deep Learning?

psgscoring is deliberately rule-based:

1. **Transparency**: Every event can be traced to a specific AASM rule and threshold. A clinician can understand *why* an event was scored.
2. **Data efficiency**: Rules require zero training data. Deep learning requires thousands of expert-scored PSGs.
3. **Auditability**: For clinical deployment (CE marking, FDA clearance), regulators require explainable decisions.
4. **Correction specificity**: Each over/under-counting mechanism is independently addressable with its own counter.

The planned hybrid approach uses a LightGBM confidence layer *on top of* the rule-based engine — not replacing it.

### 22.2 Why Not Use a Single Threshold?

A single amplitude threshold (e.g., "score hypopnea when envelope <70% of baseline") fails because:

- The baseline itself is corrupted (Fixes 1, 6)
- The envelope misses per-breath reductions (Fix 7)
- $SpO_2$ coupling has timing issues (Fixes 2, 8)
- Signal artefacts generate false events (Fix 5)

Each correction addresses a specific failure mode.

### 22.3 Performance Characteristics

| Metric | Value |
|---|---|
| Processing time (8h PSG, 256 Hz) | 3–5 min |
| Peak RAM usage | ∼2 GB |
| Event detection (apnea + hypopnea) | ∼70 s |
| All 12 corrections overhead | <1 s |

## 23. Breath Detection and Flattening Index

Individual breath detection is essential for both peak-based hypopnea detection (Fix 7) and RERA assessment (Fix 12).

### 23.1 Breath Segmentation

Breaths are detected by finding zero-crossings in the filtered flow signal. Each breath consists of an inspiratory phase (positive flow) and an expiratory phase (negative flow):

```
from psgscoring.breath import detect_breaths

breaths = detect_breaths(filtered_flow, sf=256)
# Returns list of dicts:
# [{"onset_s": 10.2, "duration_s": 3.8,
#   "amplitude": 85.3, "insp_duration": 1.6}, ...]
```

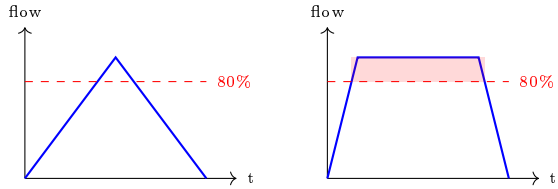
### 23.2 Flattening Index Computation

The flattening index quantifies inspiratory flow limitation. A normal breath has a triangular inspiratory waveform; a flattened breath indicates upper airway resistance:

```
from psgscoring.breath import compute_flattening_index

fi = compute_flattening_index(inspiratory_segment)
# fi ~ 0.10: normal (triangular)
# fi > 0.30: flattened (flow limitation)
# fi > 0.50: severely flattened
```

The index is defined as the fraction of inspiratory time during which the signal exceeds 80% of peak amplitude. Geometrically, this measures how "rectangular" (plateau-shaped) the inspiratory waveform is:




Normal ($FI \approx 0.10$) Flattened ($FI \approx 0.45$)

### 23.3 Breath Amplitude for Hypopnea Detection

Per-breath amplitude is compared to a local baseline (median of preceding 10 breaths):

```
from psgscoring.breath import compute_breath_amplitudes

ratios = compute_breath_amplitudes(breaths, sf=256)
# ratios[i] = breath_amplitude[i] / median(prev 10 breaths)
# ratio < 0.70 -> this breath is part of a hypopnea
```

## 24. Periodic Limb Movement (PLM) Detection

PLMs are scored per AASM 2.6 and WASM (World Association of Sleep Medicine) criteria:

- Duration: 0.5–10 seconds
- Amplitude: ≥8 $\mu$V above baseline (tibialis EMG)
- Inter-movement interval: 5–90 seconds
- A "series" requires ≥4 consecutive movements meeting the interval criterion

```
from psgscoring.plm import detect_plm

result = detect_plm(plm_left, plm_right, sf=256,
                    hypno=hypno, sleep_mask=sleep_mask)
# result["summary"]["plmi"]   -> PLM index (/h sleep)
# result["summary"]["n_plm"] -> number of PLMs in series
```

The PLM index (PLMI) = PLMs in series / hours of sleep. Clinical significance: PLMI > 15/h suggests periodic limb movement disorder (PLMD), especially if associated with arousals.

**Respiratory association:** Leg movements occurring within 0.5 s before to 0.5 s after a respiratory event are considered respiratory-associated and are *excluded* from the PLMI count. This prevents respiratory events from inflating the PLM index.

## 25. Signal Quality Assessment

Automated scoring is only as reliable as the input signals. The `signal_quality` module evaluates every channel on four dimensions:

| Check | Threshold | Method |
|---|---|---|
| Flat-line | >10% of recording | Rolling 2 s SD below role-specific threshold (0.5 $\mu$V for EEG, 0.1 mV for respiratory) |
| Clipping | >1% of samples | Signal at ADC rail for >10 consecutive samples |
| Line noise | >50% of power | 49–51 Hz and 59–61 Hz power as fraction of total 1–Nyquist power |
| Disconnect | Any occurrence | Transitions from active signal to flat-line lasting ≥10 s |

Each channel receives a grade: **good**, **acceptable**, **poor**, or **unusable**.

### 25.1 Montage Plausibility

Cross-correlation checks detect hardware errors:

- EEG ↔ EOG correlation >0.95: possible shared-reference or channel duplication

- Thorax ↔ Abdomen correlation >0.98: possible duplication (prevents obstructive/central distinction)

## 26. Common Pitfalls for Developers

### 26.1 Variable Scoping

Local variables computed in outer functions must be passed explicitly as parameters to inner functions. In Python, closures capture variables by reference, not by value. The original codebase had a bug where `_tecg`, `_r_peaks`, and `_sf_ecg_local` were local variables in `detect_respiratory_events()` but referenced inside `_detect_apneas()` — causing `NameError` in certain code paths.

### 26.2 Docker Caching

`docker compose restart` does *not* rebuild images. After modifying Python files, always use:

```
docker compose build --no-cache
docker compose up -d
```

### 26.3 Shell Escaping

Bash interprets `$` in bcrypt hashes. When writing passwords to databases, use Python's `sqlite3` with parameterised queries:

```
cursor.execute("UPDATE users SET password=? WHERE id=?",
               (hashed_pw, user_id))
```

### 26.4 EDF Limitations

- `pyedflib` is the reliable EDF+ writer; `edfio` silently writes plain EDF headers and drops annotations.
- MNE's EDF export fails on large physical min/max values.
- SomnoMedics stores position as ADC values (0–255), not integer codes — requires automapping.

### 26.5 Translation Function Shadowing

Never name local variables `t` when `t()` is imported as a translation function:

```
from i18n import t
# ...
t = date.today()    # BREAKS ALL t() CALLS!
today = date.today() # correct
```

## 27. Glossary

| Term | Definition |
|---|---|
| AHI | Apnea-Hypopnea Index: (apneas + hypopneas) / hours of sleep |
| OAHI | Obstructive AHI: only obstructive + mixed events |
| CAI | Central Apnea Index |
| RDI | Respiratory Disturbance Index: AHI + RERA index |
| RERA | Respiratory Effort-Related Arousal: flow limitation + arousal, no desaturation |
| ODI | Oxygen Desaturation Index: desaturations $\geq$3% per hour |
| PLMI | Periodic Limb Movement Index: PLMs in series per hour of sleep |
| TST | Total Sleep Time (minutes of scored sleep) |
| TIB | Time In Bed |
| SE | Sleep Efficiency: TST / TIB $\times$ 100% |
| SOL | Sleep Onset Latency: time from lights-off to first sleep epoch |
| WASO | Wake After Sleep Onset |
| RIP | Respiratory Inductance Plethysmography (effort bands) |
| TECG | Transformed ECG: high-pass filtered + QRS-blanked ECG |
| CSR | Cheyne-Stokes Respiration |
| MMSD | Mean Magnitude of Second Derivative |
| FRI | Flow Reduction Index: events with flow reduction but no AASM criteria met |
| EDF | European Data Format (PSG file format) |
| AASM | American Academy of Sleep Medicine |
| WASM | World Association of Sleep Medicine |

## 28. Exercises

1. **Signal processing:** Generate a synthetic nasal pressure signal (12 breaths/min, amplitude 100). Apply square-root linearisation. Compare the envelope before and after linearisation. What happens to the breath amplitude distribution?

2. **Baseline sensitivity:** Create a 10-minute signal with stable breathing for 5 minutes, then a 30-second apnea followed by 4.5 minutes of recovery breathing at 150% amplitude. Plot the dynamic baseline. Where does it become inflated? Implement the 30-second recovery mask (Fix 1) and compare.

3. **Classification:** Create test inputs for `classify_apnea_type()` that trigger each of the 7 rules. Verify that your inputs produce the expected type and confidence range.

4. **Property testing:** Write a Hypothesis test that generates random `effort_ratio` (0–2) and `raw_var_ratio` (0–3) values. Verify that the classifier never crashes and always returns a valid type.

5. **$SpO_2$ coupling:** Generate a $SpO_2$ signal with a known 5% desaturation at $t = 40$ s. Test `get_desaturation()` with window sizes of 30 s and 45 s. At what circulatory delay does the 30 s window fail?

6. **Flattening index:** Generate a triangular inspiratory waveform and a rectangular one. Compute the flattening index for each. What threshold best separates normal from flattened?

7. **End-to-end:** Download a public PSG from PhysioNet (Sleep-EDF dataset). Run YASA staging, then `run_pneumo_analysis()` with all three scoring profiles. Compare the AHI across profiles and explain the differences.

## Acknowledgments

This handbook accompanies the `psgscoring` library, which extends YASA by Raphael Vallat and Matthew P. Walker. The staging classifier was trained on SHHS and MESA data funded by the NHLBI.

```
pip install psgscoring
https://github.com/bartromb/psgscoring
https://slaapkliniek.be
```

## 29. References

### Standards and Scoring Manuals


[1] Berry RB, Brooks R, Gamaldo C, et al. *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications. Version 2.6.* American Academy of Sleep Medicine, 2020.

[2] Iber C, Ancoli-Israel S, Chesson AL, Quan SF. *The AASM Manual for the Scoring of Sleep and Associated Events: Rules, Terminology and Technical Specifications.* 1st ed. AASM, 2007.

[3] Zucconi M, Ferri R, Allen R, et al. The official World Association of Sleep Medicine (WASM) standards for recording and scoring periodic leg movements in sleep (PLMS) and wakefulness (PLMW) developed in collaboration with a task force from the International Restless Legs Syndrome Study Group (IRLSSG). *Sleep Med.* 2006;7(2):175–183. doi:10.1016/j.sleep.2006.01.001.


### Sleep Staging and YASA


[4] Vallat R, Walker MP. An open-source, high-performance tool for automated sleep staging. *eLife.* 2021;10:e70092. doi:10.7554/eLife.70092.

[5] Perslev M, Darkner S, Kempfner L, Nikolic M, Jennum PJ, Igel C. U-Sleep: resilient high-frequency sleep staging. *npj Digital Medicine.* 2021;4:72. doi:10.1038/s41746-021-00440-5.

[6] Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, Ye Q, Liu T-Y. LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems.* 2017;30:3146–3154. NeurIPS 2017.

[7] Gramfort A, Luessi M, Larson E, et al. MNE software for processing MEG and EEG data. *NeuroImage.* 2014;86:446–460. doi:10.1016/j.neuroimage.2013.10.027.

[8] Feinberg I, Floyd TC. Systematic trends across the night in human sleep cycles. *Psychophysiology.* 1979;16(3):283–291. doi:10.1111/j.1469-8986.1979.tb02991.x.


### Respiratory Scoring and Signal Processing


[9] Thurnheer R, Xie X, Bloch KE. Accuracy of nasal cannula pressure recordings for assessment of ventilation during sleep. *Am J Respir Crit Care Med.* 2001;164(10):1914–1919. doi:10.1164/ajrccm.164.10.2010113.

[10] Montserrat JM, Farré R, Ballester E, Felez MA, Pastó M, Navajas D. Evaluation of nasal prongs for estimating nasal flow. *Am J Respir Crit Care Med.* 1997;155(1):211–215. doi:10.1164/ajrccm.155.1.9001310.

[11] Lee H, Park J, Kim H, Lee KJ, Heo C. Detection of apneic events from single-channel nasal airflow using 2nd derivative method. *Physiol Meas.* 2008;29:N37–N45. doi:10.1088/0967-3334/29/5/N01.

[12] Hosselet J, Norman RG, Ayappa I, Rapoport DM. Detection of flow limitation with a nasal cannula/pressure transducer system. *Am J Respir Crit Care Med.* 1998;157(5):1461–1467. doi:10.1164/ajrccm.157.5.9708008.

[13] Uddin MB, Chow CM, Su SW. A novel algorithm for automatic diagnosis of sleep apnea from airflow and oximetry signals. *Physiol Meas.* 2021;42:015001. doi:10.1088/1361-6579/abd47a.

[14] Nakano H, Tanigawa T, Furukawa T, Nishima S. New rule-based algorithm for real-time detecting sleep apnea events. *J Clin Sleep Med.* 2016;12(10):1389–1396. doi:10.5664/jcsm.6200.

### Apnea Classification and Effort Assessment

[15] Berry RB, Albertario CL, Guttman B, et al. Use of a transformed ECG signal to detect respiratory effort during apnea. *J Clin Sleep Med.* 2019;15(11):1653–1660. doi:10.5664/jcsm.7880.

[16] Berry RB, Budhiraja R, Guttman B, et al. Use of chest wall electromyography to detect respiratory effort during polysomnography. *J Clin Sleep Med.* 2016;12(9):1239–1244. doi:10.5664/jcsm.6122.

### Scoring Variability and Validation

[17] Rosenberg RS, Van Hout S. The American Academy of Sleep Medicine inter-scorer reliability program: sleep stage scoring. *J Clin Sleep Med.* 2013;9(1):81–87. doi:10.5664/jcsm.2350.

[18] Rosenberg RS, Hirshkowitz M, Engleman HM, Penzel T. The AASM inter-scorer reliability program: respiratory events. *J Clin Sleep Med.* 2014;10(4):447–454. doi:10.5664/jcsm.3630.

[19] Ruehland WR, Rochford PD, O'Donoghue FJ, Pierce RJ, Singh P, Thornton AT. The new AASM criteria for scoring hypopneas: impact on the apnea hypopnea index. *Sleep.* 2009;32(2):150–157. doi:10.1093/sleep/32.2.150.

[20] Malhotra A, Younes M, Kuna ST, Benca R, Kushida CA, Walsh J, Hanlon A, Staley B, Pack AI, Pien GW. Performance of an automated polysomnography scoring system versus computer-assisted manual scoring. *Sleep.* 2013;36(4):573–582. doi:10.5665/sleep.2548.

### Novel Metrics and Complementary Approaches

[21] Parekh A, Kam K, Wickramaratne S, et al. Ventilatory burden as a measure of obstructive sleep apnea severity is predictive of cardiovascular and all-cause mortality. *Am J Respir Crit Care Med.* 2023;208(11):1216–1226. doi:10.1164/rccm.202301-0109OC.

### Epidemiology

[22] Peppard PE, Young T, Barnet JH, Palta M, Hagen EW, Hla KM. Increased prevalence of sleep-disordered breathing in adults. *Am J Epidemiol.* 2013;177(9):1006–1014. doi:10.1093/aje/kws342.

[23] Benjafield AV, Ayas NT, Eastwood PR, et al. Estimation of the global prevalence and burden of obstructive sleep apnoea: a literature-based analysis. *Lancet Respir Med.* 2019;7(8):687–698. doi:10.1016/S2213-2600(19)30198-5.

### Software and Statistical Tools

[24] Vallat R. Pingouin: statistics in Python. *J Open Source Software.* 2018;3(31):1026. doi:10.21105/joss.01026.